Super Admin User Handbook

> The complete, feature-by-feature guide to the DotStark Super Admin control plane — how to log in, what every page does, how to use every feature, and how the security model protects you while you work.

App: **frontend/super-admin** — http://localhost:3003 (dev) API: **backend-super** — http://127.0.0.1:8006 (dev)

---

1. What is Super Admin?

Super Admin is the internal operations control plane for the DotStark Booking Agent platform. Where the tenant admin panel (port 3002) lets a *customer* manage their own booking site, Super Admin lets *your staff* manage every customer organization at once:

- see all organizations, users, bookings, conversations and revenue,
- take lifecycle actions (create / suspend / restore / delete orgs),
- manage plans, subscriptions, invoices and refunds,
- run analytics reports and monitor platform health,
- toggle feature flags and send broadcast notifications,
- step into an org as a support agent and review conversations,
- keep a tamper-proof audit trail of everything done above.

Everything you do is **audited** — the audit log is append-only and cryptographically chained, so nobody (not even a database admin) can silently change history.

---

2. Logging in

2.1 First login (MFA enrollment)

1. Open http://localhost:3003 and enter your email + password.
2. If you have never enrolled in two-factor authentication (TOTP), you are prompted to **enroll**:
    - The app shows a QR code and a secret key.
    - Scan the QR with an authenticator app (Google Authenticator, Authy, 1Password, etc.).
    - Enter the 6-digit code from the app to complete enrollment.
3. From then on, every login requires the current 6-digit code from your authenticator app.

If you were **already enrolled**, skip straight to entering your TOTP code.

2.2 Sessions

- Successful login sets two cookies: `super_admin_token` (access, ~15 min) and `super_admin_refresh` (refresh, 7 days).
- The app **refreshes your session automatically** while you work — you should rarely see a login page again until you log out.
- **Log out** from the sidebar/profile menu. Sessions also end if your account is disabled or the refresh token expires.

2.3 If you can't log in

Symptom	Meaning / fix
`429 Rate limit exceeded: 10 per 1 minute`	Too many login attempts from your IP in one minute. Wait ~60 seconds and retry. This limit is intentionally strict.
`429 Too many failed attempts. Try again later.`	5 failed attempts within 15 minutes — your account (and IP) is temporarily locked out. Wait 15 minutes.
`401 Invalid credentials`	Wrong email or password. After several wrong tries the lockout above kicks in.
TOTP rejected	Your device clock is skewed — sync time in the authenticator app. Codes expire after 30 seconds.

---

3. The layout

The sidebar is your map. Items appear **only if your role has the matching permission** — a Support admin, for example, never sees Plans or Billing.

Sidebar item	Permission gate	What it's for
Dashboard	`dashboard.view`	KPIs, top orgs, active alerts, recent actions
Organizations	`orgs.view`	Browse/search orgs, open the 7-tab detail view
Users	`users.view`	Cross-org user search and profiles
Plans	`plans.view`	Plan catalog, limits, assignment
Billing	`billing.view`	Subscriptions, invoices, refunds, revenue
Analytics	`reports.view`	12 report types, JSON + CSV
Monitoring	`health.view`	Dependency health overview
Alerts	`alerts.view`	Platform alert inbox
Incidents	`incidents.view`	Incident tracking
Feature Flags	`flags.view`	Toggles, canary rollouts, kill switches
Audit Logs	`audit.view`	Tamper-proof activity trail
Approvals	`orgs.delete`	Two-person approval queue
Notifications	`notifications.view`	Broadcast campaigns
Support	`support.impersonate` / `support.conversations.view`	Impersonation + conversation peek

Sidebar item	Permission gate	What it's for
Settings	`security.manage`	Branding + security policy

4. Roles & permissions

Five roles are seeded. The **owner** of the platform has everything.

Role	Permissions	Typical use
owner	38/38 — all permissions incl. `pii.unmask`, `orgs.delete`	Platform owner; the only role that can approve deletions and unmask PII
super_admin	36/38 — everything except `orgs.delete`, `pii.unmask`	Full operational control without destructive/identity rights
operations	31/38 — all ops tools, plans, billing manage; read-only analytics	Day-to-day platform operations
support	9/38 — `support.impersonate`, `support.conversations.view`, `users.view`, audit view, dashboard	Customer support; cannot touch plans/billing/org deletion; sees PII **masked**
auditor	6/38 — read-only: audit, dashboard, reports/analytics view, monitoring	Compliance / oversight

The full permission list (38)

```
dashboard.view        orgs.view             orgs.create        orgs.update
orgs.suspend          orgs.restore          orgs.delete        orgs.export
users.view            users.disable         users.enable       users.reset_password
plans.view            plans.manage          plans.assign
billing.view          billing.manage        billing.retry_payment   billing.refund
reports.view          analytics.view        analytics.export
health.view           monitoring.view       alerts.view        alerts.manage
incidents.view        incidents.manage
flags.view            flags.manage
audit.view            notifications.view    notifications.manage
support.impersonate   support.conversations.view
security.manage       ops.manage            pii.unmask
```

PII rule of thumb: by default **everything is masked** (emails, phone numbers, names). Only the owner (`pii.unmask`) sees real PII, and only while that toggle is on in a given view.

5. Dashboard

The landing page gives a one-screen health check of the platform:

- **KPI cards** — active organizations, bookings per day, MRR, error rate, conversations, AI token usage.
- **Top organizations** — most active orgs, ranked.
- **Active alerts** — currently open platform alerts (severity-coded).
- **Recent actions** — the latest entries from the audit trail.

There is nothing to *do* here — it's a monitoring view.

---

6. Organizations

6.1 List & search

- The Organizations page lists every org with name, status (active / suspended), and live counters (users, bookings, conversations).
- Use the **search box** to filter by name; the list is server-searched, so it works even with thousands of orgs.

6.2 Organization detail (7 tabs)

Click any org to open its detail page:

Tab	Contents
Overview	Business name, timezone, status, API key (masked), booking settings (same-day booking, slot cutoff, max booking days ahead, session retention), user/booking/conversation counts
Users	Everyone in the org — search, open profiles, disable/enable/reset
Usage	Current month's usage counters (bookings, AI tokens, lifecycle events)
Billing	Subscription, invoices, payment status
Integrations	Calendar (Google/Outlook) and meeting providers, WhatsApp, website URL
Audit	Every audited action involving this org
Support	Jump into impersonation or peek at conversations (see §13)

6.3 Create an org

1. Organizations → **Create Organization**.
2. Give it a name (and optionally business name, plan, limits, branding).
3. The org is created **on the tenant side through the internal bridge** — you receive the org ID and its API key (`ds_...`) immediately.

6.4 Suspend / restore

- **Suspend** an org to take its booking site offline instantly (the tenant backend rejects the org's requests while suspended). Confirm in the dialog.
- **Restore** puts it back. Both actions are audited (`org.suspend` / `org.restore`) and counted in the org's usage.

6.5 Export

Export the org list as CSV (requires `orgs.export`). Useful for reporting.

6.6 Delete an org — the two-person approval flow

Deleting an org is **deliberately not a one-click action**:

1. You request the deletion (approval type `org.delete`, expires in 24 h).
2. **You cannot approve your own request** — the system refuses self-approval.
3. A *different* admin with `orgs.delete` approves it from the **Approvals** page (they must pass **MFA step-up** first — re-enter a TOTP code).
4. The bridge then executes the real deletion on the tenant side; the approval shows status **EXECUTED**.
5. If nobody approves within 24 h, the request expires (status **EXPIRED**) and is dead — the org is untouched.

The tenant-side audit rows survive the org deletion by design, so history is never lost.

---

7. Users

- **Search** works across all organizations (email/name search).
- **Profile** shows the user's org, booking history, and login history.
- **Disable / Enable** a user to cut off their login immediately (audited: `user.disable` / `user.enable`).
- **Reset password** issues a reset OTP flow (audited: `users.reset_password`).

---

8. Plans

The plan catalog drives what each org can do.

- **Plans list** — seeded plans: Starter, Pro, Enterprise. Each has a price, billing period, and usage limits: **bookings, AI tokens, seats, storage**. A **blank limit = unlimited**.
- **Create / edit a plan** (requires `plans.manage`): duplicate plan codes are rejected (409); bad values are rejected (422).
- **Archive a plan** instead of deleting when orgs still use it — deleting an assigned plan is blocked.
- **Assign / upgrade an org** (requires `plans.assign`): pick the org and the target plan. The assignment history is kept; each org has exactly one active plan.
- The org detail shows the plan **plus the current month's usage** against its limits, straight from the usage counters.

---

9. Billing

Three tabs: **Revenue**, **Subscriptions**, **Invoices**.

9.1 Subscriptions

Create a subscription for an org (provider-agnostic — the dev environment uses a deterministic mock provider; production can use Stripe by setting `STRIPE_SECRET_KEY`). One active (non-canceled) subscription per org.

9.2 Invoices

- Issue invoices against a subscription.
- **Retry payment** on an unpaid invoice (only while it's still `open`; 409 otherwise).

9.3 Refunds — step-up + thresholds

Refunding money is the most sensitive billing action:

1. You must pass **MFA step-up**: re-enter your TOTP code (valid 5 min) right before the refund.
2. **Refunds at or below $500** (threshold `REFUND_APPROVAL_THRESHOLD_CENTS`) execute immediately with the step-up.
3. **Refunds above $500** don't execute directly — they file a `billing.refund` **approval** (24 h expiry) that a *second* admin must approve, exactly like org deletion.
4. Every refund — executed or approved — is audited.

9.4 Revenue report

`/billing/reports/revenue` (and `.csv`): MRR, ARR, churn, invoice aging. This is the single source of truth for revenue; the Analytics page links here instead of duplicating it.

---

10. Analytics (Reports)

The Analytics page serves **12 report types**. Most are live queries over tenant data — no job to run.

Report	Shows
usage	Per-org monthly bookings + AI tokens (pick a month `YYYY-MM-01`)
bookings	Last 30 days, daily + per org
conversations	Last 30 days: sessions + messages per org
ai	AI tokens vs. messages per org (current month)
api	API call volumes by org + top actions (30 days)
storage	Document chunks + bytes per org
kb	Knowledge-base chunks by source type
growth	Orgs/users created per month (12 months)
performance	Bookings 30d / 7d, active orgs, meeting-link readiness
health	Open alerts + resolved in 30 days
widget	Bookings made through embedded widgets
revenue	Redirects to Billing's revenue report

Every report returns **JSON** and a **CSV download** (the CSV endpoint). Wrong month format → 422; unknown report → 404. (Requires `reports.view`; CSV export additionally uses `analytics.export`.)

---

11. Monitoring: health, alerts, incidents

11.1 Monitoring page

`/health/overview` aggregates what matters: open/critical alerts + open incidents. `/health/dependencies` probes the tenant platform's dependencies (DB, LLM, email — and more once the tenant side exposes them) through the internal bridge.

11.2 Alerts

Platform alerts come from the tenant side (e.g. a provider failing). You can:

- **List/filter** by severity, status, component, org.
- **Acknowledge** an alert (idempotent — repeating the action doesn't double count) so everyone knows it's being handled.
- **Resolve** it once fixed. Both actions write tenant-side audit entries and control-plane audit rows (`alerts.acknowledge` / `alerts.resolve`).

11.3 Incidents

For bigger issues you can track them here: create an incident (severity `critical|major|minor`), update its status (`open → investigating → resolved`), and see who resolved it and when. Resolving records `resolved_by` / `resolved_at`. All of it is audited.

---

12. Feature flags & notifications

12.1 Feature flags

The tenant backend reads flags on **every request**, so flags act instantly:

- **Toggle** — turn a platform feature on/off. Scope it: **platform-wide**, **per-org**, or **per-plan**.
- **Canary rollout** — enable a flag for a percentage of orgs (e.g. 5% → 25% → 100%) to graduate a release safely.
- **Kill switch** — the instant platform-wide "off" for any feature. This is the emergency brake: flip it and the tenant backend stops honoring the feature within its cache TTL.
- Flag evaluations are cached briefly, so "instant" means within the TTL — seconds, not minutes.

12.2 Notifications (broadcasts)

Draft a broadcast message → **review** it → **send** to all orgs (or to staff). The full lifecycle is audited (`notifications.manage`).

---

13. Support tools

13.1 Conversation peek

Read-only access to an org's chat transcripts. **PII is masked** before anything is displayed — emails, phone numbers, names. Support roles never see raw PII, even by accident.

13.2 Impersonation

"Log in as" an org admin to see exactly what they see:

1. You must pass **MFA step-up** (re-enter your TOTP code).
2. A **30-minute impersonation token** is issued for that admin.
3. **One active impersonation session at a time** — starting a new one revokes the previous.
4. A **visible banner** shows "you are impersonating" the whole time, so you can't forget.
5. A mandatory `impersonation.start` audit entry is written — impersonation always leaves a paper trail.
6. The session dies at 30 minutes (or when revoked/expired) automatically.

Use it to reproduce a customer issue, verify settings, or help configure a site — never outside the audit trail's comfort zone.

---

14. Settings: branding & security policy

14.1 Branding (org settings)

Website URL and branding fields for orgs (also editable from the org detail Overview tab).

14.2 Security policy (requires `security.manage`)

The global security posture for admin accounts:

- **Password policy** — minimum length / complexity the platform enforces for super-admin passwords (change it here; existing weak passwords are validated against it on next change).
- **MFA policy** — MFA is mandatory for all admins; this lets you tune it.
- **IP allowlist** — restrict which IPs may access the control plane.

Any change here is audited.

---

15. Security model — how you're protected

1. **MFA (TOTP)** on every admin login, mandatory.
2. **MFA step-up** for the most sensitive actions: refunds, approvals, impersonation, bridge writes. A fresh TOTP code, valid 5 minutes.
3. **RBAC** — 38 permissions across 5 roles; every route checks them; the UI hides what you can't use.
4. **PII masking by default** — masked unless the owner explicitly unmaskes in that view.
5. **Two-person rule** — org deletion and large refunds require a *second* admin, and self-approval is impossible.
6. **Rate limits + lockout** — 10 login attempts/minute/IP, plus a 15-minute lockout after 5 failures.
7. **Tamper-proof audit** — append-only table + DB triggers block UPDATE and DELETE, and every row links to the previous one in a hash chain. The Audit Logs page verifies the whole chain on demand and flags the exact row if it's ever broken.
8. **Read-only tenant access** — the control plane reads tenant data with a role that can only SELECT; writes go through the HMAC-signed bridge.

---

16. Approvals page quick reference

The Approvals queue is where two-person rules land. One screen, three states:

Status	Meaning	Your action
`PENDING`	Waiting for a second admin	Approve (after step-up) or **Reject**
`EXECUTED`	Fully executed (org deleted / refund issued)	None — done
`REJECTED`	A second admin declined	None — request dead
`EXPIRED`	Nobody acted within 24 h	Request a fresh one

---

17. FAQ & gotchas

- **Why is the email masked on the org page?** PII is masked by default. The owner can unmask per view; support can't.
- **The refund button asks for another TOTP code.** That's step-up — by design, for your safety. It's valid for 5 minutes.
- **I approved a deletion but nothing happened.** Check the approval status: if it shows `PENDING` still, the bridge may have failed — the runbook (see below) has the exact steps. Deletions are idempotent and safe to re-approve.
- **A page says "missing permission".** Your role doesn't include it — it's normal for support/auditor roles. Ask the owner to check Settings.
- **Why does the usage report show 0 AI tokens?** AI tokens only start accumulating after telemetry is enabled (cutover) — there's no historical source. Bookings and lifecycle events were backfilled.
- **Where is the revenue report?** It lives in Billing; Analytics links to it.
- **Is my activity really logged?** Yes — login, MFA, every view of an org detail, every write action, approvals, impersonation. The chain is verified by the Audit Logs page itself.

---

18. Operational notes

- **Cutover**: some lifecycle actions show 503 "Internal bridge is disabled" until the GA cutover flips `INTERNAL_BRIDGE_ENABLED=true` on the tenant backend (see `docs/deployment/superadmin-cutover.md`). Reads always work.
- **Testing**: the E2E suites in `scripts/e2e/` double as smoke tests — the consolidated runner exercises all 43 features.
- **Support**: for incidents, follow the runbook — `docs/deployment/ superadmin-cutover.md` §4.

19. Related documents

Document	What it covers
Super Admin implementation plan	Architecture, phases, threat model
Cutover & runbooks	Going live + incident procedures
Troubleshooting	Common platform issues

Document	What it covers
`scripts/e2e/README.md`	Running the automated E2E suites